



Traslados, altas y liberación de camas

Primera evaluación parcial — Defensa de la arquitectura propuesta

● Ocupada ● Limpieza ● Disponible

Esaú Abimael de la Cruz Mendoza

Carné 1890-21-13279

Ing. Richard David Ortíz Sasvin

Sistema Hospitalario Integrado 2026

¿Quién participa y qué procesos ejecuta?

Actores autorizados y los dos casos de uso centrales del módulo

ACTORES



Administrador

Ejecuta operaciones autorizadas del módulo



Médico

Participa en traslado y alta según autorización



Enfermera

Participa en operaciones hospitalarias



Recepcionista

Opera funciones del flujo de admisión

PROCESOS



Traslado interno de paciente

- Valida cama destino disponible en el mismo tenant
- Cama origen → limpieza · cama destino → ocupada
- Admisión permanece activa; se registra bed_transfers



Alta hospitalaria

- Valida tipo de alta (voluntaria, médica, fallecimiento, traslado)
- Cierra la admisión y registra fecha/motivo de egreso
- Cama asociada → limpieza · rechaza altas repetidas

Requisitos que sostienen el diseño

Selección de requisitos funcionales y no funcionales con mayor peso arquitectónico

REQUISITOS FUNCIONALES

RF-04

El traslado ejecuta en una única transacción atómica: libera cama origen, ocupa destino, actualiza admisión y registra historial.

RF-06 / 07

El alta requiere un tipo válido y libera la cama asociada de forma atómica al cerrar la admisión.

RF-08

Una admisión ya dada de alta no puede volver a procesarse.

RF-09

Todo endpoint exige middleware de tenant, autenticación JWT y rol autorizado.

REQUISITOS NO FUNCIONALES

RNF-01

Las operaciones críticas se ejecutan dentro de transacciones de base de datos.

RNF-02

Toda lectura y escritura se filtra por tenant_id; sin cruces entre hospitales.

RNF-04

Separación estricta Presentation → Application → Domain.

RNF-05

Las reglas de dominio no dependen de HTTP, Eloquent ni SQL.

SOLID aplicado: Liskov Substitution Principle

Traslado y alta comparten el punto de partida, pero no el mismo contrato de comportamiento



ANTES

SalidaPaciente

Clase base obliga a: cerrar admisión, liberar cama y marcarla disponible de inmediato.

TrasladoPaciente no puede sustituir a la clase base sin contradecirla:

- No debe cerrar la admisión
- Necesita validar una cama destino
- La cama origen no debe pasar a disponible

El consumidor necesita condicionales por tipo concreto para corregir el comportamiento heredado — se rompe la sustituibilidad.



DESPUÉS

OperacionDisposicionPaciente

Contrato mínimo común: admisión activa, transacción, cama origen → limpieza, trazabilidad.

OperacionTraslado y OperacionAlta implementan el contrato y agregan solo sus reglas propias:

- Traslado: valida y ocupa cama destino
- Alta: valida datos y cierra la admisión
- Ninguna contradice al contrato común

El consumidor trabaja contra la abstracción común y ya no necesita identificar el subtipo para actuar correctamente.

De la solicitud HTTP a PostgreSQL

Application depende de contratos (Repository), nunca de Eloquent directamente



Presentation

`TransferController` · `DischargeController`

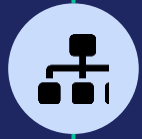
Recibe la solicitud y delega — sin SQL ni reglas de negocio



Application

`TransferPatientService` · `DischargePatientService`

Coordina el caso de uso completo dentro de una transacción



Domain

`AdmissionDischargePolicy`

Reglas de negocio independientes de HTTP y de Eloquent



Repository Interfaces

`AdmissionRepository` · `BedRepository` · `TransactionManager`

Contratos que Application conoce — no implementaciones concretas



Infrastructure

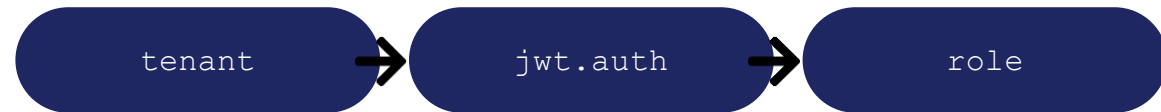
`Eloquent Repositories` · `LaravelTransactionManager` →
`PostgreSQL`

Adaptadores reemplazables sin tocar los casos de uso

Contrato API y seguridad de acceso

Dos endpoints, un mismo pasillo de seguridad antes de llegar al controlador

MIDDLEWARE



POST

`/api/v1/admissions/{admissionId}/transfer`

Delega en TransferPatientService · responde 200 con la nueva cama o 409/422 si la cama destino no está disponible.



POST

`/api/v1/admissions/{admissionId}/discharge`

Delega en DischargePatientService · responde 200 al cerrar la admisión o 409 si ya fue dada de alta.

De la regla de negocio a la prueba automatizada

Matriz decisión → evidencia: cada criterio de aceptación tiene una prueba que lo respalda

CA	Criterio	Decisión arquitectónica	Evidencia (test)
CA-01	Traslado exitoso	Transacción atómica vía TransactionManagerInterface	TransferPatientServiceTest
CA-02	Destino no disponible	Revalidación de disponibilidad en Application	TransferPatientServiceTest
CA-04	Alta repetida rechazada	AdmissionDischargePolicy, regla de dominio pura	AdmissionDischargePolicyTest
CA-05	Rollback ante fallo	LaravelTransactionManager + PostgreSQL	TransferPatientPostgreSqlTest

Matriz completa en ADR-001-arquitectura.md §10 y ESPECIFICACION.md §9–10 del repositorio.

Cambio práctico: nuevo tipo de alta



Escenario planteado: el hospital solicita un nuevo tipo de alta — “alta_administrativa” (traslado a otro centro por convenio).

1

Se agrega el valor a la lista validada en AdmissionDischargePolicy (regla de dominio, RB-08) — único punto de verdad sobre tipos de alta válidos.

2

TransferController, DischargeController, los repositorios y el contrato API no se modifican: la capa Application ya delega la validación al dominio.

3

AdmissionDischargePolicyTest se extiende con el nuevo caso; TransferPatientServiceTest y DischargePatientServiceTest quedan intactos.

Un solo punto de cambio, sin tocar HTTP, persistencia ni las reglas de traslado — consecuencia directa de la separación por capas y de aplicar LSP en el contrato de disposición del paciente.